

Yes, Ajith. **AI API SLA Monitoring Assistant** can be a very good BA project if we keep it focused on **business monitoring, SLA rules, exception handling, and stakeholder reporting** rather than building a complicated monitoring platform.

AI API SLA Monitoring Assistant

Resume-friendly project title

AI-Powered API SLA Monitoring & Business Incident Assistant

One-line description

An AI-assisted monitoring solution that analyzes API performance against defined business SLAs, identifies breaches and recurring issues, and generates actionable incident summaries for business and technical stakeholders.

1. Business problem

Imagine a company depends on several APIs:

Customer API
Payment API
Policy API
Notification API
CRM API

The business has agreed SLAs such as:

API	SLA
Customer API	99.5% availability
Payment API	99.9% availability
Policy API	Response < 2 sec
Notification API	Response < 3 sec

The current process might involve someone manually checking API reports and preparing an Excel report.

The BA/business team wants to know:

Which APIs breached SLA?

How often?

What was the business impact?

Which issues require escalation?

Which APIs are consistently underperforming?

That's what your project automates.

2. Simple process

```
API Performance Data
    ↓
AI + SLA Rules
    ↓
Compare actual vs SLA
    ↓
Identify breaches
    ↓
Classify severity
    ↓
Identify business impact
    ↓
Generate incident summary
    ↓
Generate management report
```

You don't actually need to connect to live production APIs.

Use **sample API monitoring data** to demonstrate the concept.

3. Example

Suppose your data contains:

```
API: Payment API
Date: 21-Sep-2026
Availability: 99.72%
Average Response: 2.8 sec
Error Rate: 1.4%
```

Configured SLA:

```
Availability >= 99.9%
Response Time <= 2 sec
Error Rate <= 1%
```

The system identifies:

```
SLA BREACH

Availability:
Actual: 99.72%
SLA: 99.90%

Response Time:
Actual: 2.8 sec
SLA: 2.0 sec

Error Rate:
Actual: 1.4%
SLA: 1.0%
```

4. AI adds the business analysis

Instead of simply displaying numbers, AI generates:

Business Impact

Payment API performance was below the defined SLA during the reporting period. Elevated response time and error rates may affect payment completion and could result in transaction delays or failed customer journeys.

Recommended action

Investigate payment API latency and error patterns, validate downstream payment dependencies, and review whether the issue is isolated to specific transaction types or time periods.

Notice that you're not claiming the AI knows the exact root cause. It is **interpreting the available evidence and highlighting areas for investigation**.

5. Severity classification

Define simple business rules.

Critical

↓

Availability < 99%

High

↓

Availability < SLA

AND business-critical API

Medium

↓

Response time exceeds SLA

Low

↓

Minor threshold breach

Example:

Payment API

Availability: 98.4%

→ Critical

Customer Profile API

Availability: 99.2%

SLA: 99.5%

→ Medium

The BA defines these rules based on business criticality.

6. Business impact mapping

This is the part that makes the project **BA-oriented**.

Create an API-to-business-process mapping:

API	Business Process	Criticality
Payment API	Payment Processing	Critical
Customer API	Customer Registration	High
Policy API	Policy Issuance	Critical

API	Business Process	Criticality
Notification API	Customer Communication	Medium

Now when the Payment API breaches SLA, the system doesn't just say:

"API failure."

It says:

Business Process Potentially Impacted: Payment Processing

That is a useful BA perspective.

7. AI-generated daily report

Your application can generate something like:

API SLA DAILY REPORT

21 September 2026

APIs Monitored: 12

APIs Within SLA: 9

SLA Breaches: 3

Critical:

- Payment API

High:

- Policy API

Medium:

- Notification API

Key Observation:

Payment API experienced the most significant SLA deviation and should be investigated as a priority because it supports the payment processing journey.

Recurring Issues:

- Payment API response latency increased on 3 consecutive days.
- Policy API experienced intermittent availability issues.

Recommended Follow-up:

- 1. Review Payment API latency.
- 2. Validate downstream dependencies.
- 3. Review Policy API availability trend.
- 4. Confirm whether affected transactions were recovered.

8. Dashboard

Keep it simple.

KPI cards

APIs: 12	SLA: 75%
Breaches: 3	Critical: 1

API status

API	Availability	SLA	Status
Payment	98.4%	99.9%	BREACH
Customer	99.8%	99.5%	OK
Policy	98.9%	99.5%	BREACH
Notification	99.7%	99.5%	OK

Trend

Show:

Payment API Availability over last 7 days
--

AI Insights

A separate section:

AI Observation

Payment API has breached its availability SLA on 2 of the last 7 days. The issue should be correlated with transaction failures before determining business impact.

9. Automated incident workflow

You can add one simple automation:

```
SLA Breach
  ↓
AI analyzes breach
  ↓
Severity determined
  ↓
Business process identified
  ↓
Incident summary generated
  ↓
Stakeholder notification
```

For example:

```
Payment API
98.4% availability
  ↓
SLA breach
  ↓
Critical
  ↓
Payment Processing affected
  ↓
Generate incident summary
  ↓
Notify API owner + BA/Product Owner
```

This gives you the **automation** element you were looking for.

10. BA documentation

This project should have strong BA artifacts:

```
AI-API-SLA-Monitoring/
|
├── README.md
|
├── Business-Analysis/
|   ├── Business-Problem.md
|   ├── Stakeholder-Analysis.md
|   ├── Business-Requirements.md
|   └── Functional-Requirements.md
```

11. Example user stories

US-001 — Monitor SLA

As a Business Analyst, I want API performance to be compared against defined SLA thresholds so that SLA breaches can be identified automatically.

US-002 — Identify business impact

As a Product Owner, I want SLA breaches to be mapped to affected business processes so that I can understand their potential business impact.

US-003 — Generate incident summary

As an Operations Manager, I want an automated summary of API SLA breaches so that I can quickly understand the issue and required follow-up.

12. Acceptance criteria

For US-001:

Given an API has a defined SLA

When performance data is received

Then the system should compare actual performance against the SLA.

If the actual performance falls below the SLA,

Then the API should be marked as "SLA Breach".

For US-002:

Given an API has breached its SLA

When the API is mapped to a business process

Then the system should display the affected business process and its criticality.

13. Technology

Keep this **very lightweight**:

- **Python**
- **Pandas**
- **Streamlit**
- **OpenAI API**
- **Excel/CSV**
- **Jira** — optional for incident creation

You don't need:

- ✗ **Kubernetes**
- ✗ **Kafka**
- ✗ **Elasticsearch**
- ✗ **Redis**
- ✗ **Microservices**

✗ Complex RAG

✗ Docker

The technology is there only to demonstrate the automation. **The BA analysis is the main project.**

14. Resume entry

AI-Powered API SLA Monitoring & Business Incident Assistant

Business Analyst Project

Designed an AI-assisted API SLA monitoring solution to automate performance analysis, SLA breach identification, business-impact assessment and incident reporting across API-driven business processes.

- Defined **API SLA metrics, business criticality, breach thresholds and escalation rules** for business-critical integrations.
- Mapped APIs to business processes to assess potential business impact of availability, latency and error-rate breaches.
- Designed As-Is/To-Be workflows and functional requirements for automated SLA monitoring and incident management.
- Used AI to generate **business-friendly incident summaries, observations and recommended investigation areas** from API performance data.
- Created user stories, acceptance criteria, SLA definitions, escalation matrix and requirements traceability.
- Designed management KPIs covering **SLA compliance, API availability, response time, error rate, breach frequency and incident resolution**.

This gives you a strong BA interview story:

Business Process → API Dependency → SLA → Business Rule → Automation → Exception → Business Impact → Stakeholder Reporting

And importantly, you can explain the entire project without having to pretend you are an expert backend engineer.